

E-COMMERCE DATA ENGINEERING PIPELINE

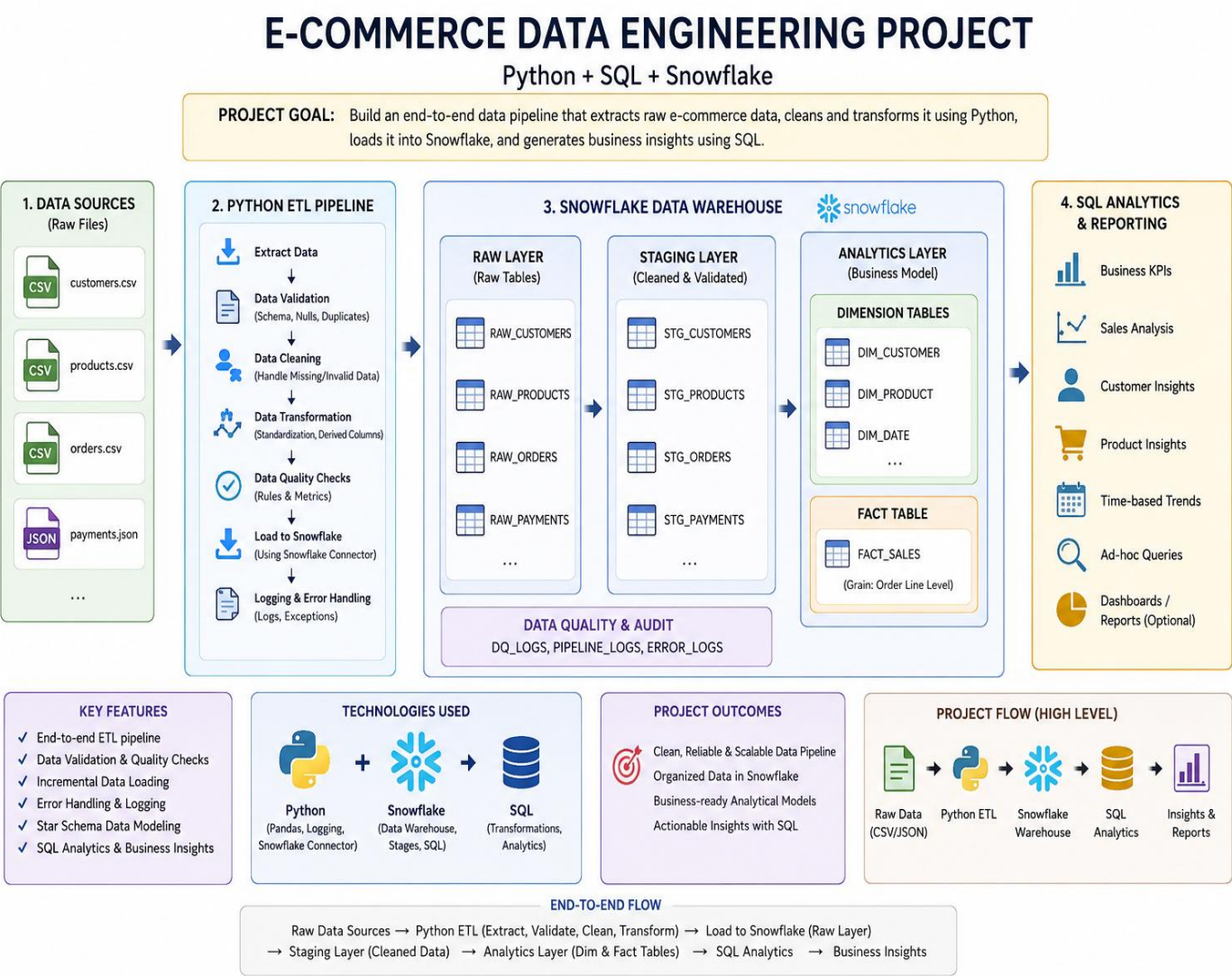
Interview-Ready Project Documentation

Python • SQL • Snowflake

Beginner-to-Interview-Level Implementation Guide

Prepared as a step-by-step build plan

~ AMEY GUJAR.



1. Project Overview

Build an end-to-end e-commerce data engineering pipeline that takes raw customer, product, and order files, validates and cleans them with Python, loads them into Snowflake, transforms them using SQL, and produces an analytics-ready dimensional model.

Project objective

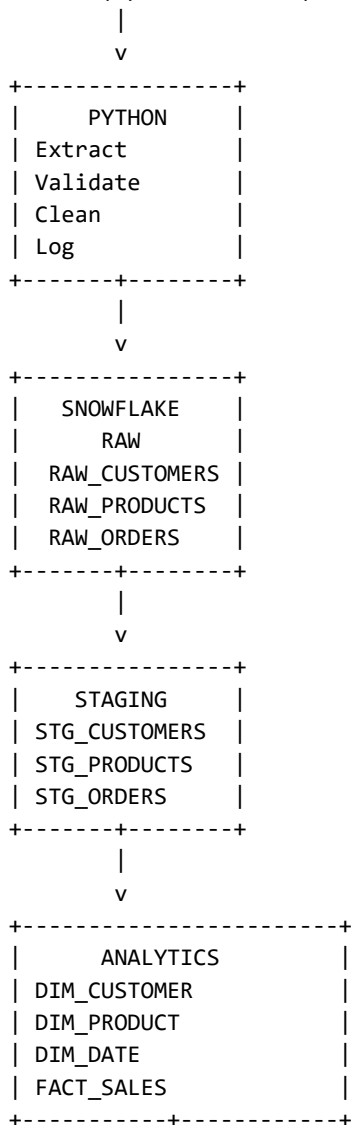
- Practice Python for extraction, validation, transformation, logging, and Snowflake loading.
- Practice Snowflake databases, schemas, stages, file formats, tables, loading, and clustering.
- Practice SQL transformations, joins, aggregations, CASE, CTEs, window functions, and data-quality checks.
- Build a project that can be explained confidently in a Data Engineer interview.

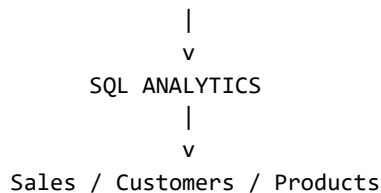
2. Final Architecture

Use the following architecture as the target design:

SOURCE FILES

customers.csv | products.csv | orders.csv





Important: build the project in phases. Do not jump directly to the final architecture.

3. Technology Stack

Technology	Purpose
Python	File ingestion, cleaning, validation, logging, Snowflake connectivity
SQL	Transformation, joins, aggregations, dimensional modeling, analytics
Snowflake	Cloud data warehouse, stages, tables, loading, analytics
CSV / JSON	Source data formats
Git/GitHub	Version control and project presentation

4. Business Requirements

The fictional company is AVD Mart, an online retailer. The pipeline must answer:

- How much revenue was generated?
- What are the top-selling products?
- Which product categories generate the most revenue?
- Which customers generate the highest revenue?
- What is monthly sales growth?
- Which customers have not purchased recently?
- Which states or cities generate the most sales?

5. Source Data Design

5.1 Customers

```
customers.csv
customer_id,customer_name,city,state,signup_date
C001,Amey,Kolhapur,Maharashtra,2025-01-10
C002,Raj,Pune,Maharashtra,2025-02-15
...
```

5.2 Products

```
products.csv
product_id,product_name,category,price
P001,Laptop,Electronics,65000
P002,Headphones,Electronics,2500
...
```

5.3 Orders

orders.csv

order_id,customer_id,product_id,order_date,quantity,discount,payment_method

01001,C001,P001,2026-01-05,1,0.05,UPI

01002,C002,P002,2026-01-06,2,0,CARD

...

6. Deliberately Include Data Problems

A realistic project should contain imperfect source data so your pipeline has something meaningful to solve.

Problem	Example	Expected handling
NULL customer_id	customer_id = NULL	Reject or quarantine record
Duplicate order	Same order_id twice	Detect duplicate
Negative quantity	quantity = -2	Reject as invalid
Bad date	2026/99/40	Flag invalid date
Missing product	Unknown product_id	Referential-integrity check
Invalid discount	discount = 1.5	Reject/flag outside allowed range
Blank values	customer_name = "	Clean or flag

7. Project Folder Structure

ecommerce-data-engineering/

```
|
|— data/
|   |— customers.csv
|   |— products.csv
|   |— orders.csv
|
|— python/
|   |— main.py
|   |— extract.py
|   |— validation.py
|   |— transformation.py
|   |— snowflake_loader.py
|   |— logger.py
|
|— sql/
|   |— 01_database_schema.sql
|   |— 02_file_formats_stages.sql
|   |— 03_raw_tables.sql
|   |— 04_staging.sql
|   |— 05_dimensions.sql
|   |— 06_fact_sales.sql
|   |— 07_data_quality.sql
|   |— 08_analytics.sql
|
|— docs/
|   |— architecture.md
|
|— README.md
```

8. Step-by-Step Implementation Plan

Phase 1 — Prepare the environment

- Create the GitHub repository and local project folder.
- Create the data, python, sql, and docs directories.
- Confirm Python and your Snowflake account/worksheet are working.
- Create a virtual environment if you use one.
- Do not write the whole pipeline yet.

Phase 2 — Prepare source data

- Create customers.csv, products.csv, and orders.csv.
- Start with a small dataset so debugging is easy.
- Add duplicates, NULLs, invalid quantities, invalid discounts, and bad dates intentionally.
- Record the expected number of valid and invalid records.

Phase 3 — Build the Snowflake foundation

- Create database AVD_ECOMMERCE.
- Create RAW, STAGING, and ANALYTICS schemas.
- Create CSV/JSON file formats as needed.
- Create an internal stage or external stage.
- Verify the current database and schema before creating objects.

Phase 4 — Load RAW data

- Create RAW_CUSTOMERS, RAW_PRODUCTS, and RAW_ORDERS.
- Load source files without business transformations.
- Run COUNT(*) and sample SELECT queries.
- Confirm raw data matches the source.

Phase 5 — Build Python ingestion and validation

- Read CSV files using Python.
- Validate required columns.
- Check data types and required fields.
- Detect duplicates.
- Check invalid quantities and discounts.
- Validate dates.
- Write valid and invalid record counts to logs.
- Use exception handling so a failure is visible and controlled.

Phase 6 — Build STAGING transformations

- Clean text fields.
- Standardize dates.
- Normalize payment-method values.
- Remove or quarantine invalid records according to your rules.
- Use SQL to transform RAW into STAGING.
- Document every transformation.

Phase 7 — Build the dimensional model

- Create DIM_CUSTOMER.
- Create DIM_PRODUCT.
- Create DIM_DATE.
- Create FACT_SALES.
- Calculate gross amount, discount amount, and net sales.
- Use surrogate keys where appropriate.

Phase 8 — Add data-quality monitoring

- Create DATA_QUALITY_LOG.
- Store run_id, table_name, total_records, valid_records, invalid_records, duplicate_records, null_records, run_timestamp, and status.
- Insert a log row after each pipeline run.
- Make failed runs easy to identify.

Phase 9 — Add incremental loading

- Choose a watermark such as order_date or ingestion timestamp.
- Load the initial historical dataset.
- Create a new file containing only new records.
- Run the pipeline again and verify that only new records are processed.
- Test duplicate prevention.

Phase 10 — Optimize Snowflake

- Identify the large fact table.
- Use a clustering key only when justified by query patterns and table size.
- Test representative filters before and after clustering.
- Document why the clustering key was chosen.

Phase 11 — Analytics SQL

- Write business queries using joins, GROUP BY, HAVING, CASE, CTEs, and window functions.
- Calculate monthly revenue.
- Find top products and customers.
- Calculate category and regional performance.
- Find inactive customers.
- Calculate month-over-month sales changes.

Phase 12 — Testing and documentation

- Test valid and invalid files.
- Test empty files.
- Test duplicate records.
- Test missing columns.
- Test Snowflake loading failures.
- Document architecture, setup, assumptions, transformations, and sample results.
- Prepare a 2–3 minute interview explanation.

9. Snowflake Object Plan

Layer	Objects	Purpose
RAW	RAW_CUSTOMERS, RAW_PRODUCTS, RAW_ORDERS	Store source-like data
STAGING	STG_CUSTOMERS, STG_PRODUCTS, STG_ORDERS	Cleaned and standardized data
ANALYTICS	DIM_CUSTOMER, DIM_PRODUCT, DIM_DATE	Dimensions for analytics
ANALYTICS	FACT_SALES	Central sales fact table
MONITORING	DATA_QUALITY_LOG	Pipeline/data-quality audit

10. Target Fact Table Logic

FACT_SALES

order_id
customer_key
product_key
date_key
quantity
unit_price
gross_amount
discount_amount
net_amount
payment_method

gross_amount = quantity * unit_price
discount_amount = gross_amount * discount
net_amount = gross_amount - discount_amount

11. SQL Interview Challenges to Implement

- Q1. Find total revenue.
- Q2. Find total number of orders and customers.
- Q3. Find the top 10 products by net revenue.
- Q4. Find revenue by category.
- Q5. Find monthly revenue.
- Q6. Find the top customer in every state using a window function.
- Q7. Find customers who have never placed an order.
- Q8. Find customers who have not purchased in the last 90 days.
- Q9. Find the month-over-month revenue change.
- Q10. Find products whose revenue is above their category average.
- Q11. Find duplicate order IDs.
- Q12. Find records with invalid discount values.
- Q13. Find orders where the customer or product does not exist.

12. Python Requirements

Module	Responsibilities
extract.py	Read CSV/JSON source files
validation.py	Schema checks, NULL checks, duplicate checks, business rules
transformation.py	Standardize and prepare data
snowflake_loader.py	Connect and load data into Snowflake
logger.py	Pipeline logs and run information
main.py	Orchestrate the complete workflow

Use environment variables or a secure secrets mechanism for Snowflake credentials. Never commit passwords or private keys to GitHub.

13. Error Handling Flow

```

START
|
v
Read source file
|
v
File exists? ---- NO ----> Log ERROR -> Stop
|
YES
v
Validate schema
|
v
Schema valid? --- NO ----> Log ERROR -> Stop
|
YES
v
Validate records
|
+--> Invalid records -> Quarantine/Log
|
v
Load valid data
|
v
Snowflake load successful?
|
NO -----> Log ERROR -> Stop/Retry
|
YES
v
Write DATA_QUALITY_LOG
|
v
SUCCESS

```


14. Incremental Loading Design

After the first full load, create a second source file containing new orders. The pipeline should identify new records and avoid loading records already processed.

Initial Run:

orders_001.csv

|

v

FULL LOAD

|

v

FACT_SALES

Next Run:

orders_002.csv

|

v

CHECK EXISTING ORDER_ID

|

+---- already exists ---> skip/log duplicate

|

+---- new -----> load

15. Testing Checklist

- ☐ All expected source columns exist.
- ☐ Required IDs are not NULL.
- ☐ Duplicate order IDs are detected.
- ☐ Quantity must be greater than zero.
- ☐ Discount must be within the agreed range.
- ☐ Dates are valid.
- ☐ Every order references an existing customer.
- ☐ Every order references an existing product.
- ☐ Raw counts match the loaded source counts.
- ☐ Staging counts are explainable after cleaning.
- ☐ Fact table revenue calculations are correct.
- ☐ Running the same file twice does not create unwanted duplicates.
- ☐ Pipeline failure is logged.
- ☐ Successful run is logged.

16. Interview Explanation

Use this structure when an interviewer asks: 'Tell me about your project.'

1. Problem:

AVD Mart receives customer, product and order data as files.

2. Solution:

I built an ETL pipeline using Python, SQL and Snowflake.

3. Python:
Used Python for extraction, validation, cleaning, logging and loading.
4. Snowflake:
Designed RAW, STAGING and ANALYTICS layers.
5. SQL:
Used joins, CTEs, aggregations and window functions for transformations and business analysis.
6. Data quality:
Handled duplicates, NULLs, invalid values and referential checks.
7. Incremental loading:
Added logic so new records can be processed without unnecessarily reprocessing the entire historical dataset.
8. Result:
Produced an analytics-ready FACT_SALES table and dimension tables.

17. Resume Version —

E-Commerce Data Engineering Pipeline | Python, SQL, Snowflake

- Developed an ETL pipeline using Python to extract, validate, clean and load e-commerce customer, product and order data into Snowflake.
- Designed RAW, STAGING and ANALYTICS layers with fact and dimension tables for analytical workloads.
- Implemented data-quality checks for duplicates, NULLs, invalid values and referential integrity, with pipeline execution logging.
- Developed incremental-loading logic to process newly arrived records while preventing duplicate ingestion.
- Created SQL transformations and analytical queries for revenue, customer, product and category performance.
-

19. Suggested Build Schedule

Day	Work
Day 1	Project setup + datasets + requirements
Day 2	Snowflake database, schemas, stages, file formats
Day 3	RAW tables + initial load
Day 4	Python extraction + validation
Day 5	Staging transformations
Day 6	Dimensions + fact table
Day 7	Data-quality logging
Day 8	Incremental loading
Day 9	SQL analytics + optimization
Day 10	Testing + GitHub README + interview preparation

21. What to Do First

Do not start by writing Python code. Start with the requirements and data model.

- Step 1: Create the project folder/repository.
- Step 2: Create the three source files: customers.csv, products.csv, orders.csv.
- Step 3: Keep the first dataset small.
- Step 4: Add a few intentional bad records.
- Step 5: Write down the expected columns and validation rules.
- Step 6: Then create the Snowflake database and schemas.

Recommended approach for learning: implement one step, test it, and only then move to the next step. This project is designed so you can build it yourself rather than copy a finished solution.